UNIVERSIDAD DE LA SABANA · PRUEBAS Y VALIDACIÓN DE SOFTWARE

Pruebas de integración y de sistema

Cinco maneras de probar la misma Registraduría, y qué ve cada una

Proyecto registraduria · JUnit · Mockito · H2 · Testcontainers · Pact · Spring Boot Test

MAPA DE LA SESIÓN

Lo que vamos a ver, y cuánto dura cada cosa

1	**Qué integra una prueba de integración** El vocabulario y el proyecto sobre el que trabajamos	8 min
2	**Integración con H2** RegistryIT: persistencia real, en memoria	10 min
3	**Dobles de prueba con Mockito** RegistryWithMockTest, y por qué no es integración	8 min
4	**Testcontainers y Pact** La base de datos de verdad, y el contrato entre servicios	14 min
5	**Ejecución, matriz y entrega** mvn verify, JaCoCo y qué se califica	10 min

POR QUÉ ESTAMOS AQUÍ

Todas las pruebas pasan y el sistema está roto

Es la situación más común en un proyecto real: **cada equipo prueba su parte, todo está en verde, y la integración falla.**

Las pruebas unitarias no mienten. Simplemente contestan una pregunta más pequeña que la que usted necesita responder antes de desplegar.

La pregunta de hoy: **¿qué pasa cuando las piezas se juntan?**



La regla de la sesión: Una prueba solo puede encontrar defectos en el código que realmente ejecuta.

VOCABULARIO

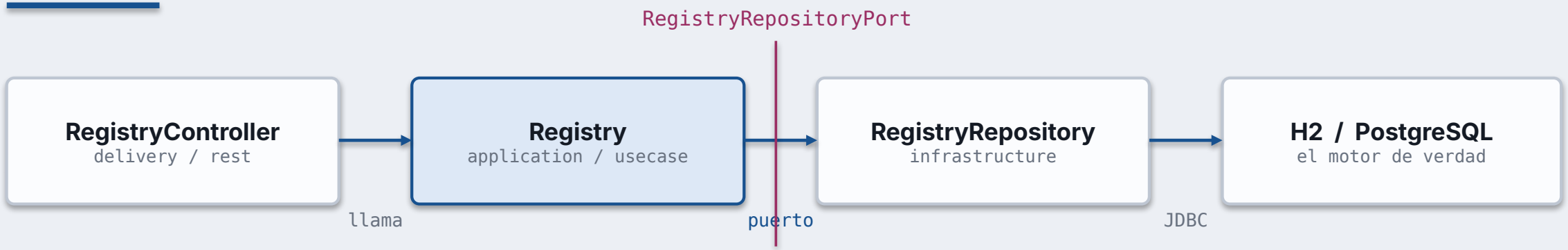
Tres pruebas, tres preguntas distintas

	Unitaria	De integración	De sistema
Qué pregunta	¿Esta clase razona bien?	¿Estas piezas se entienden entre sí?	¿El sistema se comporta como promete?
Qué ejecuta	Una clase; el resto simulado	Varias capas reales, incluida la base	Todo, entrando por la interfaz pública
Ejemplo aquí	`RegistryWithMockTest`	`RegistryIT`	`RegistryControllerIT`
Velocidad	milisegundos	décimas de segundo	segundos
Si falla, ¿dónde está?	En esa clase	En la conversación entre capas	En algún lugar del sistema

El nombre de una prueba no dice cuántas clases toca, sino cuántas fronteras cruza sin fingir ninguna.

EL PROYECTO

La Registraduría, en capas



El puerto es una interfaz, y es la bisagra de todo el taller.

`Registry` no sabe si al otro lado hay JDBC, un contenedor de PostgreSQL o un objeto de Mockito. Por eso el **mismo caso de uso** se puede probar de cinco maneras sin cambiar una línea de producción.

SIN EL PUERTO

Habría que levantar una base de datos **incluso para probar que un menor de edad no puede votar.**

Seis resultados, y el orden en que se deciden

`Registry.registerVoter`

```
public RegisterResult registerVoter(Person p) {
  if (p == null)                return INVALID;
  if (p.getId() <= 0)           return INVALID;
  if (!p.isAlive())             return DEAD;
  if (p.getAge() < 0 ||
      p.getAge() > MAX_AGE)     return INVALID_AGE;
  if (p.getAge() < MIN_AGE)     return UNDERAGE;

  if (repo.existsById(p.getId()))
                                return DUPLICATED;
  repo.save(...);               return VALID;
}
```

El orden no es cosmético.

Una edad de `-1` es un dato **imposible**; una de 17 es un dato **correcto** de alguien que todavía no puede votar.

Lo primero se corrige; lo segundo se espera. Invertir las guardas le diría a una persona con un dato mal capturado que espere a cumplir años.

Y una persona de 15 años fallecida devuelve `DEAD`, no `UNDERAGE`: la guarda de `alive` corta antes.

Para la matriz de pruebas: cada guarda es una clase de equivalencia, y cada clase de equivalencia necesita su caso de prueba.

La primera frontera real: la base de datos

RegistryIT.java

```
@Before public void setup() throws Exception {
  repo = new RegistryRepository("jdbc:h2:mem:regdb...");
  repo.initSchema();
  repo.deleteAll();
  registry = new Registry(repo);      // repositorio REAL
}

@Test public void shouldRegisterValidPerson() {
  Person p1 = new Person("Ana", 100, 30, FEMALE, true);
  RegisterResult r = registry.registerVoter(p1);

  assertEquals(RegisterResult.VALID, r);
  assertTrue(repo.existsById(100));   // vuelve a la BD
}
```

QUÉ SE EJECUTA DE VERDAD

— El caso de uso Registry

— El adaptador RegistryRepository

— El SQL: CREATE, SELECT, INSERT

— La transacción y su commit

— El motor H2, en memoria

assertEquals por sí sola no prueba que se haya guardado nada. Solo la segunda línea consulta la base.

Fíjense en que se llama RegistryIT y no RegistryTest. El nombre decide quién la ejecuta, y lo vemos al final.

Qué defecto encuentra cada aserción

assertEquals(VALID, r)
`mira el valor devuelto`

SIGUE EN VERDE SI...

— el INSERT nunca se ejecutó
— la transacción hizo rollback
— el commit falta
— la tabla tiene otro nombre

assertTrue(repo.existsById(100))
`vuelve a consultar la base`

SE PONE EN ROJO SI...

— el INSERT nunca se ejecutó
— la transacción hizo rollback
— el commit falta
— el SELECT consulta la columna equivocada

Una prueba de integración que no consulta el otro lado de la frontera no está integrando: está ejecutando código extra y afirmando lo mismo que una unitaria.

Mockito: cuando no quiere la base de datos

RegistryWithMockTest.java

```
@Before public void setUp() {
  repo = mock(RegistryRepositoryPort.class);
  registry = new Registry(repo);
}

@Test public void devuelveDuplicatedSiElRepoDiceQueExiste() {
  when(repo.existsById(7)).thenReturn(true);
  Person p = new Person("Ana", 7, 25, FEMALE, true);

  assertEquals(DUPLICATED, registry.registerVoter(p));
  verify(repo, never()).save(anyInt(), anyString(),
                             anyInt(), anyBoolean());
}
```

Cero SQL. Cero conexiones. Microsegundos.

El mock responde **lo que usted le dijo** que respondiera. Eso es su virtud y su límite en la misma frase.

La aserción cambia de naturaleza: ya no se afirma sobre un **dato**, sino sobre una **conversación** — que el caso de uso no intentó guardar.

Ojo con el nombre. Si todos los colaboradores están simulados, no se está integrando nada: es una prueba unitaria del caso de uso. Por eso el archivo se llama RegistryWithMockTest y corre en mvn test.

Lo que un mock nunca podrá ver

	Con mock (unitaria)	Con H2 (integración)
Colaborador	Doble del puerto	`RegistryRepository`
Qué verifica	Que el caso de uso colabora bien	Que los datos quedaron guardados
Detecta	Errores de lógica de negocio	Errores de SQL, esquema, tipos, transacciones
Velocidad	**milisegundos**	décimas de segundo
No puede detectar	**que el INSERT está mal escrito**	poco; pero es más lenta y frágil

Se necesitan las dos. El error común es creer que los mocks reemplazan a la integración: un mock siempre responde lo que usted le dijo, incluso si la base de datos real haría otra cosa.

H2 es rápida, pero no es su base de datos

H2 difiere de PostgreSQL en dialecto SQL, tipos, plegado de identificadores y comportamiento transaccional.

Una prueba verde contra H2 puede ocultar un fallo que solo aparece cuando el código llega al motor de producción.

RegistryRepositoryPostgresIT.java

```
@Testcontainers
class RegistryRepositoryPostgresIT {

  @Container
  static PostgreSQLContainer<?> db =
    new PostgreSQLContainer<>("postgres:16-alpine");

  // la prueba usa db.getJdbcUrl()
}
```

Testcontainers levanta un motor real dentro de un

	H2	Testcontainers
Velocidad	milisegundos	segundos
Fidelidad	aproximada	**total**
Prerrequisitos	ninguno	Docker corriendo
Cuándo	ciclo rápido mientras desarrolla	antes de integrar, y en CI

El mismo SQL, dos respuestas distintas

```
H2         ::  SELECT "name" FROM registry  ->  FALLA: no existe
POSTGRES   ::  SELECT "name" FROM registry  ->  OK, valor = Ana
```

El estándar SQL dice que los identificadores sin comillas se pliegan a una caja, pero no dice a cuál. Cada motor eligió distinto.

H2 los pliega a MAYÚSCULAS, así que la columna se llama `NAME`. PostgreSQL los pliega a minúsculas: `name`. Un identificador entrecomillado, en cambio, se toma literal.

H2 → la columna se llama NAME

PostgreSQL → se llama name

El mismo SQL no es portable. Es exactamente el tipo de defecto que una prueba con H2 nunca va a encontrar, porque para H2 el código está bien.

Caja negra: entrar por donde entra el cliente

RegistryControllerIT.java

```
@RunWith(SpringRunner.class)
@SpringBootTest(webEnvironment = RANDOM_PORT)
public class RegistryControllerIT {

  @Autowired private TestRestTemplate rest;

  @Test public void registraUnaPersonaValida() {
    ResponseEntity<String> resp = rest.postForEntity(
        "/register", new HttpEntity<>(json, headers), String.class);

    assertEquals(HttpStatus.OK, resp.getStatusCode());
    assertEquals("VALID", resp.getBody());
  }
}
```

La prueba no nombra a `Registry` ni a la base de datos. Solo conoce una URL, un JSON y una respuesta.

Los objetos ya no los arma la prueba: los arma Spring con `RegistryConfig`. Es la única de las cinco que ejercita el controlador y el mapeo del JSON.

Todo lo que puede afirmar cabe en dos líneas: el código de estado y el cuerpo.

El precio de la caja negra. Cuando falla, dice «esperaba VALID, llegó 500», sin señalar la línea culpable. Es la prueba más completa y la más cara de diagnosticar: por eso va arriba de la pirámide, no en la base.

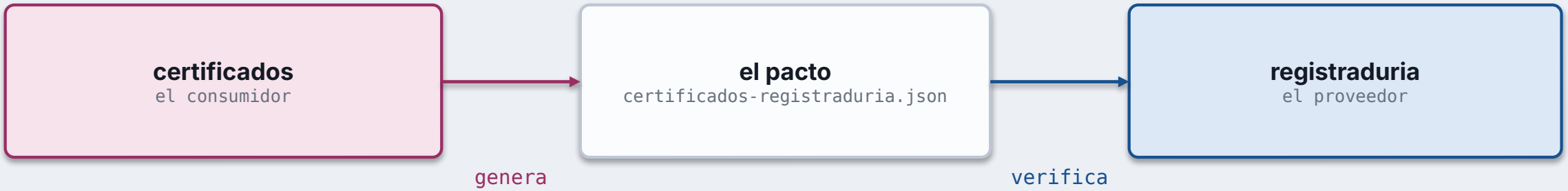
El hueco que ninguna de las anteriores cubre

La Registraduría deja de responder texto plano y empieza a responder `{"resultado":"VALID"}`. ¿Qué prueba se entera?

Prueba	¿Detecta el cambio?	Por qué
`RegistryWithMockTest`	**No**	El mock responde lo que usted le dijo, no lo que responde el servicio real
`RegistryIT (H2)`	**No**	No pasa por HTTP
`RegistryControllerIT`	**No**	Verifica al proveedor contra sí mismo: si actualiza la prueba junto con el código, sigue verde
`Pruebas del consumidor`	**Sí, pero tarde**	En su propio pipeline, o peor: en producción

El problema de los sistemas distribuidos: Cada equipo prueba su parte y todas pasan, pero el sistema completo está roto. Levantar todos los servicios juntos es lento, frágil y no escala.

El consumidor escribe el contrato



1 · LADO CONSUMIDOR

Declara qué espera del proveedor y corre sus pruebas contra un servidor simulado que se comporta así. Como subproducto, genera el archivo de pacto.

Aquí: `CertificadoServicePactTest` emite el

2 · LADO PROVEEDOR

Toma ese pacto y verifica, en su propio pipeline, que sigue cumpliéndolo. Reproduce todas las interacciones que el consumidor declaró.

Aquí: `RegistraduriaProviderPactIT` levanta la aplicación y responde

Los dos servicios nunca se levantan al mismo tiempo, y aun así el acuerdo queda verificado.

Quién nombra el estado y quién sabe montarlo

consumidor: nombra el estado que necesita

```
.given("ya existe un votante con id 901")
.uponReceiving("un registro duplicado")
.path("/register").method("POST")
.willRespondWith()
.status(200).body("DUPLICATED")
```

proveedor: sabe cómo montarlo

```
@State("ya existe un votante con id 901")
void conVotante901() {
  registry.registerVoter(
      new Person("Luis", 901, 40, MALE, true));
}
```

El consumidor no sabe que hay una tabla, y no debe saberlo. Pide un estado por su nombre; el proveedor decide si eso significa un INSERT, un contenedor o un archivo. Ese desacople es el punto entero del contract testing.

Compruébelo usted mismo: 1. mvn clean verify, todo en verde. 2. En RegistryController cambie return r.name() por un JSON. 3. mvn verify: RegistryControllerIT puede seguir pasando, pero el pacto falla.

El nombre del archivo decide qué se ejecuta

validate | **compile** | **test** | package | integration-test | verify

lo que hace mvn test

mvn verify llega hasta aquí

Archivo	Plugin que lo ejecuta	mvn test	mvn verify
RegistryWithMockTest.java	Surefire (*Test.java)	**corre**	**corre**
RegistryIT.java	Failsafe (*IT.java)	**no corre**	**corre**
RegistryRepositoryPostgresIT.java	Failsafe (*IT.java)	**no corre**	**corre**
RegistryControllerIT.java	Failsafe (*IT.java)	**no corre**	**corre**

El falso verde más común: Si usa mvn test para verificar su trabajo verá BUILD SUCCESS sin haber probado ninguna integración. Y JaCoCo necesita sus cuatro ejecuciones: sin las de integración, lo que cubren los *IT no cuenta y el 80% global es inalcanzable.

Qué prueba atrapa qué defecto

Defecto	Mock	H2	Testcont.	HTTP	Pact
Guardas de edad en el orden equivocado	sí	sí	sí	sí	no
El SELECT consulta la columna equivocada	no	sí	sí	sí	no
save() nunca confirma la transacción	no	depende	depende	depende	no
Identificador SQL que H2 pliega distinto	no	no	sí	no	no
Gender en minúsculas revienta el mapeo	no	no	no	sí	no
La respuesta cambia de texto plano a JSON	no	no	no	no	sí

Léala por columnas. Ninguna columna gana siempre, y ninguna fila la atrapan todas. Por eso el taller pide las cinco técnicas y no la mejor. Esta tabla, con sus casos reales, es uno de los entregables.

Qué se califica

01 Repositorio
Rama principal que compile con mvn clean verify, .gitignore e integrantes.

02 Wiki
Documentación con enlaces a cada clase y a cada test.

03 Pruebas con H2
Al menos tres, en formato AAA, con aserciones que consulten la base.

04 Pruebas con mocks
Al menos dos, usando when, verify y never correctamente.

05 Pruebas de sistema
Al menos dos por HTTP, cubriendo respuesta correcta y error.

06 Cobertura
JaCoCo combinado: target/site/jacoco y jacoco-it.

07 Matriz de pruebas
Caso, entrada, resultado esperado, tipo y test que lo valida.

08 Gestión de defectos
defectos.md con al menos un defecto real, con evidencia.

09 Reflexión final
Qué capa costó más probar, y qué aporta cada técnica.

La rúbrica premia que las pruebas verifiquen persistencia real, no que sean muchas.

PARA LLEVARSE

Cuatro frases

Una prueba solo encuentra defectos en el código que ejecuta de verdad.
Todo lo demás es confianza, no verificación.

Un mock responde lo que usted le dijo.
Es rapidísimo y por eso mismo no puede contradecirle.

Si no consulta el otro lado de la frontera, no está integrando.
assertEquals sobre el valor devuelto no prueba que se haya guardado nada.

El nombre del archivo decide si su prueba corre.
*Test.java en mvn test; *IT.java solo en mvn verify.

Complemento visual interactivo del taller: `pruebas_integracion_visual.html` en el repositorio